

BSUID — Plan de migración (WhatsApp Business-Scoped User ID)

Contenido	
TL;DR	
1. Contexto técnico	
1.1 Qué es el BSUID	
1.2 Timeline (verificado)	
1.3 Cómo llega (y NO llega) el BSUID a este servicio	
1.4 Outbound a un usuario BSUID-only	
1.5 Fuentes	
2. Glosario	
3. Análisis de impacto en el código (validado contra el repo)	
3.1 Identidad y creación de usuario	
3.2 BFF: identificación por teléfono — ● riesgo de crash confirmado	
3.3 Outbound (Twilio)	
3.4 Blacklist	
3.5 Push proactivo y notificaciones	
3.6 Sesión / contexto del usuario en Redis (clave derivada de source_id)	
3.7 Observabilidad	
3.8 BFF: contexto bancario	
3.9 Admin lookup	
3.10 Estado actual de soporte BSUID en el código	
4. Estrategia	
5. Plan de acción por etapas (de menor a mayor impacto)	
Etapa 0 — Defensas + observabilidad (AHORA, no-regret)	
Etapa 1 — Outbound-ready (Twilio, barato y aditivo)	
Etapa 2 — Captura BSUID inbound (● gated en Chatwoot #13837)	
Etapa 3 — Identificación sin teléfono (post-Chatwoot)	
Etapa 4 — Blacklist por BSUID (post-Chatwoot)	
Etapa 5 — Push proactivo por BSUID (post-Chatwoot + coordinación ●)	
6. QA, canary y rollback	
6.1 Test suite	
6.2 Canary (para Etapa 2+ cuando aplique)	
6.3 Rollback	
7. Riesgos y mitigaciones	
8. Preguntas abiertas	
9. Próximos pasos	
Apéndice A — Payloads de referencia	
A.1 Twilio webhook BSUID-only (referencia; NO es nuestra superficie de inbound)	
A.2 Chatwoot webhook (esperado — pendiente spec oficial de #13837)	
Apéndice B — Mapping de campos por canal	
Apéndice C — Cambios respecto del Draft 1	

BSUID — Plan de migración (WhatsApp Business-Scoped User ID)

Status: Draft 2 — Revisado contra documentación oficial de Twilio y contra el código real del repo
Owner: José Orellana Fecha: 2026-06-17 Reemplaza a: Draft 1 (2026-05-28) — corrige la estrategia “Twilio-first”, que partía de una premisa falsa
Target ventana de cambio: junio 2026 en adelante (usernames ya en rollout)
Documentos relacionados: MESSENGER_ARCHITECTURE.md

WhatsApp empezó a hacer rollout en **junio 2026** de la feature de **usernames**, que permite al cliente final ocultar su número de teléfono al business. Cuando un cliente la activa, el `wa_id` / `phone_number` puede dejar de venir en los webhooks y el identificador del cliente pasa a ser el **BSUID** (Business-Scoped User ID): identificador estable por par `(business_portfolio, customer)`, formato `whatsapp:CC.<alphanumeric>`, hasta 128 caracteres alfanuméricos (140 con el prefijo `whatsapp:`).

Tres hechos verificados que definen este plan:

- 1. **Contact Book reduce el impacto inmediato.** Meta lanzó (early abril 2026) una *contact book* que almacena automáticamente los pares `(teléfono ↔ BSUID)` de interacciones previas. **Todo cliente que alguna vez nos escribió mantiene su teléfono en los webhooks aunque active username.** El tráfico genuinamente BSUID-only se limita a contactos **nuevos** que activan username **antes** de su primer mensaje — una fracción pequeña del volumen en el corto plazo para una base instalada como la del banco.
- 2. **Chatwoot es el único portón de entrada y todavía no soporta BSUID.** En este servicio **todo el inbound entra por Chatwoot** (`WhatsApp → Twilio → Chatwoot → Pampa`). **No recibimos webhooks directos de Twilio ni de Meta.** Mientras Chatwoot no exponga el BSUID ([issue #13837](#), sin PR ni timeline), una conversación BSUID-only llega con `sender.phone_number` vacío y **no tenemos forma oficial de leer el BSUID** — ni para identificar al cliente ni para responderle.
- 3. **Twilio outbound ya está desbloqueado.** Para *enviar* a un usuario BSUID-only, la regla oficial de Twilio es literal: “Pass the `ExternalUserId` value everywhere your integration currently expects a phone number.” No requiere refactor: solo no anteponer `+` y dejar pasar el prefijo `whatsapp:`. Pero esto solo sirve una vez que **tenemos** el BSUID, que solo llega por el inbound (Chatwoot).

Conclusión estratégica: el único trabajo verdaderamente urgente y no-regret es la **Etapla 0 (defensas + observabilidad)**, que evita crashes y degrada elegante para el poco tráfico BSUID-only que pueda aparecer hoy. Todo el resto (identidad, blacklist, push completos por BSUID) está **gated sobre Chatwoot** y debe esperar a tener (a) soporte de Chatwoot o (b) métrica de volumen que lo justifique. Esto encaja con el freeze pre-launch: parches puntuales y aditivos ahora, refactors después.

Esfuerzo pre-launch realista: **Etapla 0 + Etapla 1 ≈ 3-5 eng-days**, todo retrocompatible.

1. Contexto técnico

1.1 Qué es el BSUID

Meta introduce el **Business-Scoped User ID** como identificador durable de cliente dentro del contexto de un business portfolio. Cuando el cliente activa username, el BSUID reemplaza al teléfono como *primary key* del cliente para nuestra integración.

Propiedad	Valor (verificado en doc oficial Twilio)
Formato	<code>whatsapp:CC.<alphanumeric></code> (ej. <code>whatsapp:AR.1A2B3C4D5E ...</code>). <code>CC</code> = código de país de 2 letras.
Formato parent (portfolios linkeados)	<code>whatsapp:CC.ENT.<alphanumeric></code> (ej. <code>whatsapp:BR.ENT.1A2B ...</code>)

Propiedad	Valor (verificado en doc oficial Twilio)
Longitud máxima	128 caracteres alfanuméricos (sin contar <code>CC.</code>). Campo completo con prefijo <code>whatsapp:</code> = máximo 140 caracteres.
Scope	Por par (<code>business_portfolio</code> , <code>customer</code>)
Estabilidad	Estable mientras el cliente no cambie de número de teléfono
Regeneración	Meta regenera el BSUID si el cliente cambia de número (→ se ve como “usuario nuevo” para nosotros)
Compartimentación	El mismo cliente en otro business portfolio tiene otro BSUID
Reverso	No es posible derivar el teléfono del BSUID
Presencia en webhooks	El campo <code>ExternalUserId</code> viene en TODOS los webhooks de Twilio , tenga o no el usuario activado username

1.2 Timeline (verificado)

Fecha	Evento	Impacto para nosotros
early abril 2026	Meta lanza la Contact Book : guarda pares (<code>phone</code> ↔ <code>BSUID</code>) de interacciones previas	● Clientes conocidos mantienen teléfono aunque activen username → blast radius chico
junio 2026	WhatsApp inicia rollout de usernames a usuarios finales	<code>wa_id</code> / <code>from</code> pueden venir vacíos; primeras conversaciones BSUID-only (solo contactos nuevos)
junio 2026 → 2026 H2	El campo BSUID pasa a ser requerido en los payloads de webhook	Twilio ya lo manda como <code>ExternalUserId</code> ; Chatwoot todavía no lo expone
	Adopción progresiva del feature	Crece lento el % de tráfico BSUID-only

Nota: hoy es **2026-06-17** — estamos dentro de la ventana de rollout. La Etapa 0 no es preventiva a futuro: es para el tráfico que puede empezar a llegar esta semana.

1.3 Cómo llega (y NO llega) el BSUID a este servicio

1.3.1 Realidad verificada: todo entra por Chatwoot

Validado contra el código: **el inbound entra exclusivamente por** `app/routes/chatwoot_events.py` (`POST /webhook/events`), que invoca `app/controllers/chatwoot_controller.py` → `message_received()`.

- **No existe** ningún handler que parsee `From` / `To` / `ExternalUserId` / `MessageSid` de Twilio.
- El `TwilioMessenger` es **solo outbound** (envío).
- El bridge real es: `WhatsApp` → `Twilio` → `Chatwoot` → `Pampa`.

Por lo tanto, **el BSUID solo puede entrar a este servicio el día que Chatwoot lo exponga en su payload**. Hasta entonces:

```
// Chatwoot, conversación BSUID-only (hoy, sin soporte BSUID)
{
  "event": "message_created",
  "sender": {
    "phone_number": null,          // o "" o malformado
    "identifier": null,          // ● Chatwoot todavía NO lo popula con el BSUID
    "type": "contact"
  }
}
```

● **Bloqueante externo:** [chatwoot#13837](#). Sin PR ni timeline. Su modelo `ContactInbox.source_id` tiene unique constraint en DB y espera E.164 → necesitan schema migration de su lado (y tolerar 140 chars).

1.3.2 Twilio (referencia — no es nuestra superficie de inbound)

Para entender qué *podría* mandar Chatwoot cuando integre, así es como Twilio expone el BSUID. La regla de población oficial:

```
# Con teléfono visible:
From=whatsapp:+5491165324855
ExternalUserId=whatsapp:AR.1A2B3C4D5E ...      # el BSUID viaja en paralelo

# BSUID-only (cliente con username, sin teléfono):
From=whatsapp:AR.1A2B3C4D5E ...                # los tres campos = BSUID
To=whatsapp:AR.1A2B3C4D5E ...
ExternalUserId=whatsapp:AR.1A2B3C4D5E ...
```

Regla oficial: “If a phone number is present, `to` / `from` contain only the phone number, and `ExternalUserId` contains the BSUID. If no phone number is present, Twilio populates `to`, `from`, and `ExternalUserId` with the BSUID.”

1.3.3 Meta Cloud API directo

No aplica. No recibimos webhooks de Meta directamente. Los campos `user_id` / `from_user_id` de Meta Cloud API no son nuestra superficie de integración; solo importan en cuanto Chatwoot/Twilio los traduzcan.

1.4 Outbound a un usuario BSUID-only

Regla oficial de Twilio: “Pass the `ExternalUserId` value everywhere your integration currently expects a phone number.” Es decir, enviamos el BSUID en el mismo campo `to` que hoy usamos para el teléfono, con prefijo `whatsapp:` y **sin** anteponer `+`.

Excepción documentada: “All message types are supported except one-tap, zero-tap, and copy-code authentication templates, which require a phone number.” → esos templates **no funcionan** para BSUID-only y necesitan plan B (ej. OTP por canal alternativo).

1.5 Fuentes

- [Twilio Changelog — WhatsApp usernames & BSUID](#) (oficial)
- [Twilio Docs — Key Concepts for WhatsApp Business Platform](#) (oficial)
- [Chatwoot Issue #13837 — BSUID support](#)
- [Vonage — Understanding WhatsApp Usernames and BSUIDs](#)

2. Glosario

Término	Definición
BSUID	Business-Scoped User ID. Identificador alfanumérico estable por par <code>(business_portfolio, customer)</code> .
wa_id	WhatsApp ID legacy = número E.164 sin <code>+</code> . Puede venir vacío post-junio 2026.
ExternalUserId	Campo Twilio que contiene el BSUID en todos los webhooks (presente o no el teléfono).
Contact Book	Feature de Meta (abril 2026) que guarda pares <code>(phone ↔ BSUID)</code> de interacciones previas; mantiene el teléfono visible para clientes conocidos.
session_id (interno)	Identificador interno: <code>f"{source_id}-{inbox_id}"</code> . El <code>source_id</code> podría pasar a ser BSUID el día que Chatwoot lo provea.
identity_mode (propuesto)	Discriminador en el modelo <code>User</code> : <code>"phone"</code> o <code>"business_id"</code> .
bff_client_id	Identificador de cliente del BFF bancario (remoto). Único identificador de negocio real hoy en el sistema.

3. Análisis de impacto en el código (validado contra el repo)

Mapeo de los puntos que asumen la existencia del teléfono. **Estado** indica la validación contra el código real al 2026-06-17.

3.1 Identidad y creación de usuario

Punto	Acoplamiento	Estado	Acción
<code>chatwoot_controller.py:186-199</code> <code>searching_user</code>	<code>body["sender"]</code> <code>["phone_number"].replace('+549', '')</code> y <code>original_phone_number</code>	✅ Exacto	Tolerar <code>None</code> /vacío; capturar <code>sender.identifier</code> cuando Chatwoot lo exponga
<code>user_repository.py:22-29</code> <code>create_user</code>	<code>create_user(user_id, username, user_phone=None, ...)</code>	✅ Exacto	Agregar parámetro <code>bsuid=None</code>
<code>user_model.py:39</code>	<code>telefonoEmisor: Optional[str] = None</code>	✅ Exacto (ya bien tipado)	Agregar <code>business_scoped_user_id: Optional[str] = None</code> e <code>identity_mode</code>
<code>user_model.py:118</code>	<code>self.telefonoEmisor = "264555666777"</code>	⚠️ Código debug en <code>build_dynamic_data()</code> (no corre en prod)	Borrar o ignorar; no es crítico

3.2 BFF: identificación por teléfono — 🚫 riesgo de crash confirmado

Punto	Acoplamiento	Estado	Acción
<code>user_model.py:166</code>	<code>"telefono": ''.join(filter(str.isdigit, self.telefonoEmisor))</code>	🚫 <code>TypeError</code> si <code>telefonoEmisor</code> <code>is None</code>	Guard inmediato (Etapa 0). Bug #1.

Punto	Acoplamiento	Estado	Acción
identification_nodes_step_1.py:715-729 <code>search_client_by_phone_number_user</code>	<code>bff_service.get_client_by_phone_number(phone_number=user.telefonoEmisor, ...)</code>	✓ Exacto	Short-circuit: si <code>not phone_number</code> → <code>bff_user_exists=False</code> sin llamar al BFF
user_identity.py:48-77 <code>ensure_user_identity</code>	invoca el lookup por teléfono	✓ Exacto	Mismo guard; log de baja severidad, no error
chatwoot_controller.py:249 <code>/restart</code>	<code>get_client_by_phone_number(user.telefonoEmisor, 'ACTIVO')</code>	✓ Exacto	Skip si no hay teléfono; avisar al operador
chatwoot_controller.py:255-275 <code>/change_number(_force)</code>	parsea <code>new_phone_number</code> del mensaje	✓ Exacto	Aceptar BSUID como input alternativo (post-Chatwoot)

3.3 Outbound (Twilio)

Punto	Acoplamiento	Estado	Acción
message_router.py:662-677 <code>_get_user_phone_number</code>	lee <code>original_phone_number</code> → <code>telefonoEmisor</code>	✓ Exacto	Devolver <code>None</code> si <code>identity_mode=="business_id"</code> y no hay teléfono → cae al fallback
message_router.py:679-695 <code>_fallback_to_chatwoot</code>	camino de fallback existente	✓ Exacto	Sirve tal cual para BSUID-only
twilio_messenger.py:155-173 <code>_normalize_phone_number</code>	antepone <code>+</code> siempre	✓ Exacto	No anteponer <code>+</code> si el valor es BSUID; dejar pasar <code>whatsapp:</code>
twilio_messenger.py:288-298 <code>send_text_message</code>	requiere <code>to_phone</code> no vacío	✓ Exacto	Aceptar BSUID en el mismo campo
<code>twilio_messenger.py</code> templates auth	one-tap/zero-tap/copy-code	—	No soportan BSUID (regla oficial) → marcar rechazables para BSUID-only
chatwoot_events.py:602-619 macro biométrica	<code>if not telefono: return 400 missing_phone</code>	✓ Exacto	No rechazar solo por falta de teléfono si hay BSUID; outbound por fallback

3.4 Blacklist

Punto	Acoplamiento	Estado	Acción
chatwoot_controller.py:97-123	<code>blacklist_repo.is_blacklisted(original_phone_number, ...)</code>	✓ Exacto	Pasar también <code>bsuid</code> y <code>chatwoot_contact_id</code>
blacklist_repository.py:11,38,67	<code>_normalize_phone</code> , <code>is_blacklisted</code> , atributo <code>telephone</code>	✓ Exacto	Agregar atributo <code>businessScopedUserId</code> ; firma multikey
Tabla <code>pampa-blacklist</code>	PK + atributo <code>telephone</code>	—	DynamoDB es schemaless → no requiere migration estructural; sí actualizar lecturas

3.5 Push proactivo y notificaciones

Punto	Acoplamiento	Estado	Acción
redis_connection.py:61-63 <code>set_notification_session</code>	<code>key = f"{user.telefonoEmisor}-{user.data['dni']}"</code>	✓ Exacto — clave hardcodeda al teléfono	Doble-persistir <code>BSUID: {bsuid}-{dni}</code> durante transición
chatwoot_controller.py:69-71	setea push solo si <code>user.telefonoEmisor</code> and <code>dni</code>	✓ Exacto	Setear si <code>(phone OR bsuid) AND dni</code>
Productor de eventos bancarios (externo)	despacha por teléfono	●	Coordinar contract change: aceptar DNI + opcional BSUID

3.6 Sesión / contexto del usuario en Redis (clave derivada de `source_id`)

El contexto completo del usuario se cachea en Redis (TTL 12h). A diferencia de la key de notificación, **esta clave NO se construye desde `telefonoEmisor`** sino desde el `source_id` que provee Chatwoot.

Cadena real verificada:

- [chatwoot_controller.py:194-197](#): `chat_id = body["conversation"]["contact_inbox"]["source_id"]` → `session_id = f'{chat_id}-{channel_id}'`
- [user_repository.py:22-23](#): `create_user(session_id, ...)` → `User(chat_id=session_id)`
- [redis_connection.py:21-46](#): `update_user_session` / `get_user_session` usan `user.chat_id` como clave → **clave final** = `{source_id}-{channel_id}`

Punto	Acoplamiento	Estado	Acción
redis_connection.py:21-46 <code>update_user_session</code> / <code>get_user_session</code>	<code>clave = user.chat_id = {source_id}-{channel_id}</code>	● Self-healing: hoy <code>source_id = wa_id</code> (contiene el teléfono); el día que Chatwoot mande el BSUID como <code>source_id</code> , la clave pasa a <code>{BSUID}-{channel_id}</code> sin cambios de código	Ninguna en la mecánica de la clave. Sí validar que el payload tolere <code>telefonoEmisor=None</code> (lo tolera — solo se serializa)
Payload <code>get_class_as_dict</code> (user_model.py:102-110)	guarda <code>telefonoEmisor</code> , <code>original_phone_number</code>	✓ No crashea con BSUID-only (quedan <code>None</code>)	Ninguna

Diferencia clave vs la key de notificación (3.5): la sesión de contexto sigue transparentemente lo que Chatwoot ponga en `source_id`, así que migra sola. La key de notificación está hardcodeda a `telefonoEmisor` y sí necesita el parche de doble-persistencia. **El riesgo de esta sección no es un crash sino la continuidad de identidad** — ver §7.

3.7 Observabilidad

Punto	Acoplamiento	Estado	Acción
chatwoot_controller.py:59-66	<code>set_tag("phone_number", ...)</code> , <code>set_user(...)</code>	✓ Exacto	Agregar tag <code>bsuid</code> ; fallback de <code>set_user</code> a BSUID

Punto	Acoplamiento	Estado	Acción
agent.py:231	outbox bancario manda "telephone": telefonoEmisor	✅ Exacto	Agregar campo bsuid; aceptar telephone=None

3.8 BFF: contexto bancario

Punto	Acoplamiento	Estado	Acción
user_model.py:163-166 create_user_session	manda telefono filtrado al BFF	❌ mismo crash que 3.2	Guard; permitir telefono=""; coordinar campo bsuid con BFF

3.9 Admin lookup

Punto	Acoplamiento	Estado	Acción
bff_route.py:892, :1079	get_client_by_phone_number(phone_number=local_phone, ...)	⚠️ Doc original citaba líneas 640-687 (incorrecto — eso es el callback de purchase_authorization). Las búsquedas reales están en ~892 y ~1079.	Agregar lookup por chat_id / session_id / chatwoot_contact_id

3.10 Estado actual de soporte BSUID en el código

Búsqueda exhaustiva: **no existe hoy** ninguno de business_scoped_user_id, identity_mode, ExternalUserId, from_user_id, BSUID. El único identificador de negocio es bff_client_id (remoto). El helper get_user_business_id propuesto **no existe todavía** — y ojo: derivar source_id de chat_id devuelve el wa_id/teléfono actual, **no** el BSUID. El BSUID será None hasta que Chatwoot lo provea.

4. Estrategia

La matriz "A/B/C" del Draft 1 quedó obsoleta: la "Opción A — Twilio-first" asumía que recibíamos webhooks de Twilio, lo cual **no es cierto**. Con los hechos verificados, hay una sola línea coherente:

- Etapas 0 + Etapa 1 ahora** (≈3-5 eng-days, todo aditivo y retrocompatible): defensas, degradación elegante, observabilidad y outbound listo. Esto cubre el poco tráfico BSUID-only que pueda aparecer durante el rollout, **sin** romper nada del flujo phone-only existente.
- Etapas 2 en adelante: gated sobre Chatwoot #13837**. No se empieza el código hasta que (a) Chatwoot libere soporte BSUID, o (b) la métrica de la Etapa 0 muestre volumen BSUID-only que lo justifique.
- Contingencia (Opción C)**: parsear additional_attributes /raw passthrough de Chatwoot detrás de un feature flag, **solo si** aparece volumen antes que el release oficial de Chatwoot. Frágil — última opción.

Esto respeta el freeze pre-launch (parches puntuales/aditivos ahora; refactors grandes a post-launch) y minimiza el blast radius porque la Contact Book mantiene el teléfono para los clientes conocidos.

5. Plan de acción por etapas (de menor a mayor impacto)

Esfuerzo en **engineering-days** (dev senior con familiaridad del repo). Las etapas están ordenadas para minimizar riesgo: cada una es independiente, retrocompatible y desplegable por separado.

#	Etapas	Esfuerzo	Bloqueante	Cuándo
0	Defensas + observabilidad	2-3	—	Ahora (pre-launch)
1	Outbound-ready (Twilio)	1-2	—	Ahora (pre-launch)
2	Captura BSUID inbound	3-5	● chatwoot#13837	Cuando Chatwoot libere / aparezca volumen
3	Identificación sin teléfono	1	depende de Etapa 2	Post-Chatwoot
4	Blacklist por BSUID	2-3	proc. ops	Post-Chatwoot
5	Push proactivo por BSUID	5-8	● productor eventos	Post-Chatwoot + coordinación
QA	E2E + canary + runbook	3-5	—	Acompaña 1-5

Etapas 0 — Defensas + observabilidad (AHORA, no-regret)

Objetivo: que ninguna conversación BSUID-only crashee el servicio, que degrade elegante, y que podamos **medir** cuánto tráfico BSUID-only real tenemos.

Criterio de aceptación:

- Ningún path lanza `TypeError` / `AttributeError` cuando `telefonoEmisor is None`.
- Una conversación con `phone_number` vacío fluye sin error (sin BFF match, outbound por fallback Chatwoot).
- Métrica de blast radius disponible: conteo de conversaciones con `sender.phone_number` vacío/nulo.
- Sentry tagueo `bsuid` (puede ser `None` por ahora).

Cambios:

1. Guard del crash en `user_model.py:163-166`:

```
telefono_clean = ""
if self.telefonoEmisor:
    telefono_clean = "".join(filter(str.isdigit, self.telefonoEmisor))
session_info = bff_service.create_user_session(
    self.conversation_id,
    self.data["session_id"],
    {"documento": self.data.get("dni"), "genero": self.data.get("genero"), "telefono":
telefono_clean},
)
```

2. Campos nuevos en el modelo `user_model.py:39`:

```
business_scoped_user_id: Optional[str] = None
identity_mode: Literal["phone", "business_id"] = "phone"
```

3. Helper de identidad en `user_identity.py` (abstracción de transición):

```

from typing import Literal, Optional
IdentityMode = Literal["phone", "business_id"]

def get_user_phone_or_none(user) → Optional[str]:
    return getattr(user, "telefonoEmisor", None) or None

def get_identity_mode(user) → IdentityMode:
    mode = getattr(user, "identity_mode", None)
    if mode in ("phone", "business_id"):
        return mode
    return "phone" if get_user_phone_or_none(user) else "business_id"

```

4. **Skip BFF lookup sin teléfono** en [identification_nodes_step_1.py:715](#) y [user_identity.py:48](#):

```

if not phone_number:
    user.data['bff_user_exists'] = False
    logger.info("[IDENTIFICATION] Skip BFF lookup: sin teléfono (BSUID-only)")
    return

```

5. **Macro biométrica** en [chatwoot_events.py:602](#): no rechazar solo por falta de teléfono si hay BSUID (por ahora siempre será `None` → comportamiento idéntico, pero deja el hook listo).

6. **Sentry** en [chatwoot_controller.py:59-66](#):

```

sentry_sdk.set_tag("phone_number", phone_number or "")
sentry_sdk.set_tag("bsuid", user.data.get("business_scoped_user_id", ""))
sentry_sdk.set_user({"id": user.chat_id,
    "username": username or phone_number or user.data.get("business_scoped_user_id") or
    "unknown"})

```

7. **Métrica de blast radius**: en `searching_user / message_received`, contar (log estructurado + tag Sentry) las conversaciones donde `sender.phone_number` viene vacío/nulo. **Es la métrica que decide si vale la pena invertir en las Etapas 2-5.**

8. **Doble-persistencia Redis** (barata, aditiva) en [redis_connection.py:61](#):

```

def set_notification_session(self, user, state):
    try:
        dni = user.data.get("dni")
        if not dni:
            return
        bsuid = user.data.get("business_scoped_user_id")
        keys = []
        if user.telefonoEmisor:
            keys.append(f"{user.telefonoEmisor}-{dni}")
        if bsuid:
            keys.append(f"BSUID:{bsuid}-{dni}")
        for key in keys:
            self.client.set(key, state, ex=NOTIFICATION_TTL)
    except Exception as err:
        logger.error(f"Error creating notification session: {err}")

```

Riesgo: ninguno; todos los cambios son retrocompatibles. Con `bsuid` siempre `None` hoy, el comportamiento es idéntico al actual salvo el guard del crash.

Etapa 1 — Outbound-ready (Twilio, barato y aditivo)

Objetivo: que el día que tengamos un BSUID, el envío funcione sin tocar nada más.

Criterio de aceptación:

- `_get_user_phone_number` devuelve `None` para `identity_mode=="business_id"` sin teléfono → fallback Chatwoot.
- `_normalize_phone_number` acepta BSUID (no antepone `+`).
- one-tap/zero-tap/copy-code lanzan error claro si se invocan para BSUID-only.

Cambios:

1. [message_router.py:662](#):

```
def _get_user_phone_number(self, user) → Optional[str]:
    if getattr(user, "identity_mode", "phone") == "business_id":
        return None # BSUID-only → fallback Chatwoot
    if getattr(user, "original_phone_number", None):
        return user.original_phone_number
    if getattr(user, "telefonoEmisor", None):
        return user.telefonoEmisor
    return None
```

2. [twilio_messenger.py:155](#) — regla oficial “usar el BSUID donde iría el teléfono”:

```
def _normalize_phone_number(self, identifier: str, prefix: str = "whatsapp:") → str:
    s = str(identifier)
    if s.startswith(prefix):
        return s
    # BSUID (CC.alphanumeric): NO anteponer "+"
    if not s.startswith("+") and "." in s and s.split(".", 1)[0].isalpha():
        return f"{prefix}{s}"
    if not s.startswith("+"):
        s = f"+{s}"
    return f"{prefix}{s}"
```

3. Marcar templates de autenticación (one-tap/zero-tap/copy-code) como no soportados para BSUID-only con error explícito.

Nota: esta etapa no entrega valor hasta tener un BSUID (Etapa 2), pero es barata y deja el camino limpio.

Etapa 2 — Captura BSUID inbound (● gated en Chatwoot #13837)

Objetivo: capturar y persistir el BSUID cuando Chatwoot lo exponga.

Pre-requisito: [chatwoot#13837](#) liberado, o decisión de activar la contingencia (Opción C) por volumen.

Criterio de aceptación:

- Toda conversación con BSUID en el payload persiste el valor en `user.data["business_scoped_user_id"]`.
- Si no hay teléfono → `identity_mode="business_id"`, `telefonoEmisor=None`.
- Tests unitarios: phone+bsuid / bsuid-only / phone-only.

Cambios (cuando Chatwoot libere):

1. `chatwoot_controller.py:186` `searching_user`:

```
sender = body.get("sender", {}) or {}
phone_raw = sender.get("phone_number", "") or ""
bsuid = sender.get("identifier") or sender.get("user_id") # campo exacto a confirmar contra el
PR de Chatwoot
phone_number = phone_raw.replace("+549", "") if phone_raw else None
original_phone_number = phone_raw or None
# ... propagar bsuid en el return y al create_user
```

2. Propagar `bsuid` a `create_user`.

Contingencia (Opción C): parsear `additional_attributes.user_id` o el raw passthrough detrás de `FF_BSUID_CHATWOOT_RAW_PARSE`. Desactivable si Chatwoot cambia su shape interno.

Etapas 3 — Identificación sin teléfono (post-Chatwoot)

Objetivo: evitar llamadas inútiles al BFF y dejar que el `identification_agent` pida DNI manual.

Criterio de aceptación: `search_client_by_phone_number_user` retorna `bff_user_exists=False` sin llamar al BFF si no hay teléfono (ya cubierto por el guard de Etapa 0); el greeting no asume `first_name` no vacío.

La mayor parte ya queda hecha en Etapa 0; esta etapa solo afina el flujo conversacional para pedir DNI.

Etapas 4 — Blacklist por BSUID (post-Chatwoot)

Objetivo: bloqueo efectivo para clientes sin teléfono visible.

Criterio de aceptación:

- `pampa-blacklist` soporta `businessScopedUserId` como criterio (DynamoDB schemaless — solo cambian las lecturas).
- Firma multikey:

```
def is_blacklisted(self, phone_number=None, business_id=None,
                    chatwoot_contact_id=None, assistant_code=None) → bool:
    if not (phone_number or business_id or chatwoot_contact_id):
        return False
    # combinar filtros con OR; mantener lógica de assistant_code
```

- Procedure ops documentado para alta sin teléfono (operador conoce solo `chatwoot_contact_id` / `bsuid`).

Etapas 5 — Push proactivo por BSUID (post-Chatwoot + coordinación 🟡)

Objetivo: que los push banco → cliente lleguen a usuarios BSUID-only.

Bloqueante: contract change del productor de eventos bancarios. Sin él, los clientes BSUID-only **nunca** reciben push.

Sub-fases:

- 5.1 Doble persistencia** — ya cubierta en Etapa 0 (`phone-dni` + `BSUID:bsuid-dni`).

2. **5.2 Endpoint de resolución** — `GET /push/session?dni=...` que devuelva las keys activas para un DNI. El productor consulta antes de despachar.
3. **5.3 Contract change del productor** — acepta `dni` + `bsuid` opcional. Para clientes BSUID-only nuevos, el productor recibe el `bsuid` desde core bancario.

Acción inmediata: iniciar la conversación con el equipo del productor **ya** (lead time de terceros), aunque el código espere.

6. QA, canary y rollback

6.1 Test suite

Tipo	Cobertura	Herramienta
Unit	helpers de identidad, guards, parsers de payload	pytest
Integration	<code>MessageRouter</code> con <code>identity_mode="business_id"</code>	pytest + Twilio mock
E2E	conversación BSUID-only → respuesta IA → outbound por fallback	Postman / collection nueva
Regression	conversaciones phone-only existentes siguen idénticas	suite actual

Recordatorio (memoria del proyecto): no ejecutar pytest automáticamente salvo pedido explícito. Tests escritos por el equipo; ejecución manual.

6.2 Canary (para Etapa 2+ cuando aplique)

Feature flag `FF_BSUID_ROUTING`, rollout escalonado: 0% (solo Etapa 0) → 5% → 25% → 100%. Métricas en Sentry/LangSmith:

- `TypeError` / `AttributeError` con tag `bsuid`.
- Tasa de fallback a Chatwoot (sube progresivamente).
- Tasa de identificación exitosa (baja levemente — más usuarios sin BFF match).
- % de conversaciones con `phone_number` vacío** (la métrica clave de blast radius).

6.3 Rollback

- Etapa 0-1:** retrocompatibles; no requieren rollback.
- Etapa 2:** rollback = dejar de persistir `bsuid` / apagar `FF_BSUID_CHATWOOT_RAW_PARSE`.
- Etapa 4-5:** backup de tabla blacklist y schema versionado de keys Redis.

7. Riesgos y mitigaciones

Riesgo	Pro b.	Imp act o	Mitigación
Conversación BSUID-only crashea el servicio hoy	Me dia	Alt o	Etapa 0 ya (guard <code>TypeError</code>)

Riesgo	Prob.	Impacto	Mitigación
Chatwoot tarda meses en liberar BSUID	Alta	Medio	Etapas 0/1 cubren el corto plazo; Contact Book limita el blast radius; Opción C en standby
Volumen BSUID-only crece antes que el release de Chatwoot	Baja - Medio	Medio	Métrica de Etapa 0 lo detecta; activar Opción C si pasa un umbral
Productor de eventos bancarios no se adapta	Medio	Alto	Iniciar coordinación YA; doble-persistencia como buffer
Auth templates (one-tap) críticos para flujo bancario	Medio	Alto	Auditar qué flujos los usan; preparar OTP por canal alternativo
Cliente cambia de número → BSUID regenerado → “usuario nuevo”	Alta	Bajo	Documentar como limitación conocida; no mitigable de nuestro lado
Pérdida de continuidad de contexto al transicionar phone→BSUID: cambia el <code>source_id</code> → nueva clave Redis (<code>{source_id}-{channel_id}</code>) → se pierde la sesión/contexto cacheado (TTL 12h) y el usuario arranca “de cero” (ver §3.6)	Medio	Bajo - Medio	Contact Book mantiene el teléfono para clientes conocidos → no transicionan. Los BSUID-only nuevos arrancan ya como BSUID, sin contexto previo que perder. No requiere acción de código; documentar
Campo destino del BSUID no tolera 140 chars	Baja	Medio	Validar longitud de columnas/keys al persistir

8. Preguntas abiertas

1. ¿Qué % del tráfico llega hoy con `phone_number` vacío? → lo responde la métrica de Etapa 0. Decide la inversión en Etapas 2-5.
2. ¿El BFF bancario va a aceptar BSUID como identificador? Necesario para `create_user_session` y push.
3. ¿El productor de eventos bancarios es de este equipo o externo? Determina el lead time de Etapa 5.
4. ¿Usamos authentication templates (one-tap/copy-code) en algún flujo bancario? Si sí, necesitan plan B con BSUID-only.
5. ¿Hay un mapping DNI → BSUID que podamos exponer al productor? Si no existe, la Etapa 5.2 requiere diseño nuevo.
6. ¿Qué campo exacto usará Chatwoot para el BSUID (`sender.identifier`, `additional_attributes`, `source_id`)? Confirmar contra el PR de #13837.

9. Próximos pasos

1. ☐ Compartir este doc con tech lead y equipo de producto.
2. ☐ Aprobar arranque de **Etapas 0 + Etapa 1** (3-5 eng-days, no-regret).
3. ☐ Desplegar la **métrica de blast radius** (Etapa 0, punto 7) y dejarla corriendo 1-2 semanas.
4. ☐ Suscribirse a [chatwoot#13837](#) para enterarse del release.
5. ☐ Iniciar conversación con el equipo del productor de eventos (Etapa 5) — lead time.

- 6.  Confirmar con BFF bancario soporte BSUID.
- 7.  Decidir el umbral de volumen que dispara Etapas 2-5 / Opción C.

Apéndice A — Payloads de referencia

A.1 Twilio webhook BSUID-only (referencia; NO es nuestra superficie de inbound)

```
MessageSid=SMxxx
From=whatsapp:AR.1A2B3C4D5E6F7G ...
To=whatsapp:+541112345678
Body=hola
ExternalUserId=whatsapp:AR.1A2B3C4D5E6F7G ...
ProfileName=Juan
```

A.2 Chatwoot webhook (esperado — pendiente spec oficial de #13837)

```
{
  "event": "message_created",
  "message_type": "incoming",
  "sender": {
    "id": 12345,
    "name": "Juan",
    "phone_number": null, // vacío en BSUID-only
    "identifier": "AR.1A2B3C4D5E6F7G ... ", //  ESPERADO – campo exacto SIN confirmar
    "type": "contact"
  },
  "conversation": {
    "id": 67890,
    "contact_inbox": { "source_id": "AR.1A2B3C4D5E6F7G ... ", "inbox_id": 1 }
  }
}
```

Nota: la forma exacta del payload Chatwoot post-BSUID **no está confirmada**. Validar contra el PR de #13837 cuando exista.

Apéndice B — Mapping de campos por canal

Campo lógico	Twilio	Chatwoot (esperado)	Variable en <code>User</code>
Identificador estable (post-BSUID)	<code>ExternalUserId</code>	<code>sender.identifier</code> (sin confirmar)	<code>business_scoped_user_id</code>
Identificador estable (pre-BSUID)	<code>From</code> (<code>whatsapp:+E164</code>)	<code>sender.phone_number</code>	<code>telefonoEmisor</code>
Teléfono visible	<code>From</code> (E164 con <code>+</code>)	<code>sender.phone_number</code>	<code>original_phone_number</code>
Identidad por canal	(sin equivalente)	<code>contact_inbox.source_id</code>	<code>chat_id</code> (= <code>source_id-inbox_id</code>)

Campo lógico	Twilio	Chatwoot (esperado)	Variable en <code>User</code>
ID de contacto en plataforma	(n/a)	<code>sender.id</code>	<code>data["chatwoot_contact_id"]</code>

Apéndice C — Cambios respecto del Draft 1

Tema	Draft 1	Draft 2 (este doc)
Vía de inbound	Asumía webhooks Twilio directos posibles	Verificado: todo por Chatwoot; Twilio solo outbound
Estrategia	“Opción A — Twilio-first” recomendada	Eliminada (premisa falsa). Etapa 0/1 ahora + resto gated en Chatwoot
Contact Book	Mencionada al pasar	Reposicionada: reduce el blast radius inmediato (clientes conocidos mantienen teléfono)
Longitud BSUID	128 chars	128 alfanuméricos + 140 con prefijo <code>whatsapp:</code>
Outbound a BSUID	Rama de detección de formato compleja	Simplificado: regla oficial “usar el BSUID donde iría el teléfono”; no anteponer <code>+</code>
Auth templates	“authentication templates” genérico	Precisado: one-tap / zero-tap / copy-code
<code>bff_routes</code> admin	Líneas 640-687	Corregido a ~892 y ~1079 (640-687 es <code>purchase_authorization</code>)
Crash <code>TypeError</code>	Rango 163-167	Línea exacta 166 (user_model.py:166)